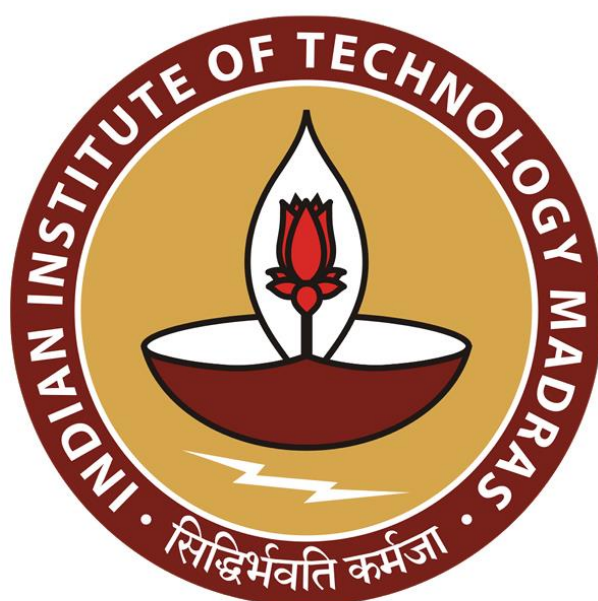


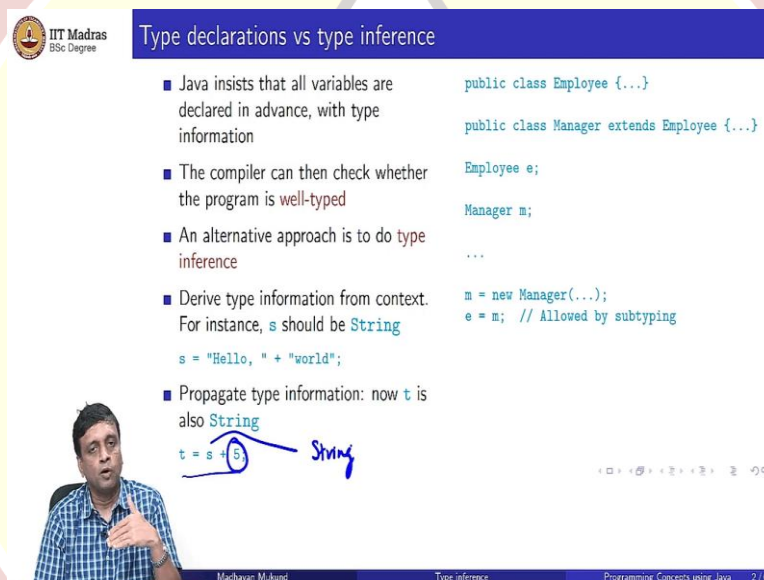


IIT Madras
ONLINE DEGREE

Programming Concepts Using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Week 8
Type Inference

So, right from the beginning, we have been talking about Java as a strongly typed language and we have been spending a lot of time about declaring types and using them consistently. So, let us look at a topic called Type Inference, which is quite popular in programming languages.

(Refer Slide Time: 00:31)



Type declarations vs type inference

- Java insists that all variables are declared in advance, with type information
- The compiler can then check whether the program is well-typed
- An alternative approach is to do type inference
- Derive type information from context. For instance, `s` should be `String`
`s = "Hello, " + "world";`
- Propagate type information: now `t` is also `String`
`t = s + s;`

```
public class Employee {...}
public class Manager extends Employee {...}

Employee e;
Manager m;
...

m = new Manager(...);
e = m; // Allowed by subtyping
```

Madhavan Mukund

So, we have type declarations and we want to contrast each type inference. So, what Java announce. It tells us to do, is to declare all variables in advance with their type. And this type information is then used later on. So, for instance, suppose we have these two public classes Manager and Employee, we could say that `e` is a variable that is of type Employee and `m` is a variable that is of type Manager.

So, this tells Java, the compiler, that if any value is assigned to `e`, it must be compatible with Employee. If any value is assigned to `m` it must be compatible with Manager. So, it allows Java to check that certain assignments and updates are compatible with the types that are defined.

So, now, if I write something which seemingly looks mistyped like for example if I assign m, sorry to be a sorry, if I assign m to be a Manager which is fine and then I allow e to point to m, it looks like I am taking an Employee variable and pointing it at a variable which is different. But, because we know that Manager extends Employee by the sub typing relation, every time I want to see an Employee I am willing to take a Manager. So, this is allowed.

So, these are the kinds of type checks that Java will do, statically, when compiling the program, to make sure that all these left-hand sides and right-hand side of the assignment statements are consistent with the types which you have declared for the variables. So, this is what is known as a well-typed program.

So, well-typed program is one in which all the expressions, all the assignments, everything as far as the Java compiler can tell, do not violate any typing. And the reason that this is important is because many problems that arise in programming are actually because of poor typing. People take an object of one type and try to use it in some other context without realizing it. It might be an accident or it might be misunderstanding of the object.

So, actually catching typing errors eliminates a large number of mistakes in programming. So, it is important to do it early and do it well at compile time. So, the alternative to this is that you actually try to infer the type; you try to figure out whether the program is well typed. You do not depend on the programmer to annotate.

So, what we are asking so far is the programmers must announce, they must write down the types and you check that the types they have written down are compatible with the way they are using it. The other way of looking at it is saying, if this is the way that you are using the type, then can I make sure that the type is sensible.

So, this is what is called Type Inference. So, for example supposing I write an expression like this, the string "Hello" plus the string "world". Now, this as a pair of string constants is a well-defined expression in Java and it returns a string, right. You take two strings and you concatenate them you get back a string.

Now, I assign it to a variable s. So, if this is legal, so this is the now though you reverse the argument. The earlier argument was given the types, is this legal? Now, for type inference you are saying if this is legal, what should be the type? So, you are trying to infer the type, you are trying to deduce the type from the information that you have. It is like a detective

story. Now, you are trying to, it is like a puzzle. You are saying this is what the programmer has said, now, can I associate a type with this?

So, here you would say for example, that okay, this is what the programmer has said and this is what the programmer intends, then that programmer probably requires this `s` to be a string, so let me assume that `s` is a string and proceed. So, this is what type inference does. So, at this point the Java compiler or whatever compiler is using type inference, would know that `s` is a string.

So, now I proceed and I write `t=s+5`. So, now plus is ambiguous, because plus could be integer addition, it could be floating point addition, it could be and in particular, one of the arguments here is an integer. So, what is the type of `t` will? On its own, this does not tell us. But now, I have already inferred that `s` is a string and remember that we have seen for example in many of the expressions that if you take a string and add something then implicitly the second thing is converted to a string.

We see it in `printf`, `println` statements and so on. So, now, Java will now take this type inference one step further and saying, okay, I already have figured out that `s` is a string, so therefore now `t` must be a string. So, in this way you can propagate this inference as you go along.

(Refer Slide Time: 5:00)

IIT Madras BSc Degree

Type inference

- Assume code is well-typed, derive most general types
 - Use information from constants to determine type
`s = "Hello, " + "world";`
 - Propagate type information based on already inferred types
`t = s + 5;`
- More ambitious?
 - If `x.bonus()` is legal, `x` must be `Manager` rather than `Employee`
- Keep track of and validate type obligations

```
public class Employee {...}
public class Manager extends Employee {
    ...
    public double bonus (...) {...}
}
...
public static f(Employee x){
    ...
    double d = x.bonus(...);
    // x must be a Manager?
    ...
}
```

Madhavan Mukund

3 / 6

So, really the goal of type inference is to work backwards. You assume the code is well typed and you try to figure out what the types should be, so that the entire program holds together in some sense. There are no inconsistencies. So, the first case, we said is that you can take

constants and expressions involving constants and use that to determine the types. So, here because we had a concatenation of two strings, we were able to deduce that `s` is a string.

And now we take something that we already know. We already know that `s` is a string. So, we can now extend our inference by propagating this information. We can take the type information. So, it is an, it is a kind of incremental process. But notice that this is all being done by reading the code that is the critical thing. It is not being done by waiting for the execution.

So, what more can we do? Well, let us get back to our Employee, Manager thing. So, we have an Employee and supposing Manager extends Employee and adds extra functionality in the form of method called `bonus` which is not there in Employee. Now, we have a function here which takes an Employee but inside this function, it calls `bonus`.

Now, there are two ways of doing it. The traditional Java compiler will say, I have no way of knowing that this Employee `x` is actually a Manager and therefore, I will assume that this is wrongly typed and I will throw an error. Whereas the type inference mechanism will say, oh, you believe that this thing has a `bonus` defined.

So, maybe every time this function is called, even though you have declared `x` to be only of type Employee, maybe every time this function is called, `x` is always a Manager. So, what I will assume in this code henceforth is, that `x` must be a Manager and see if that works. So, this would be an even more ambitious thing saying, depending on what functions you are invoking, you can distinguish between an object and a subclass.

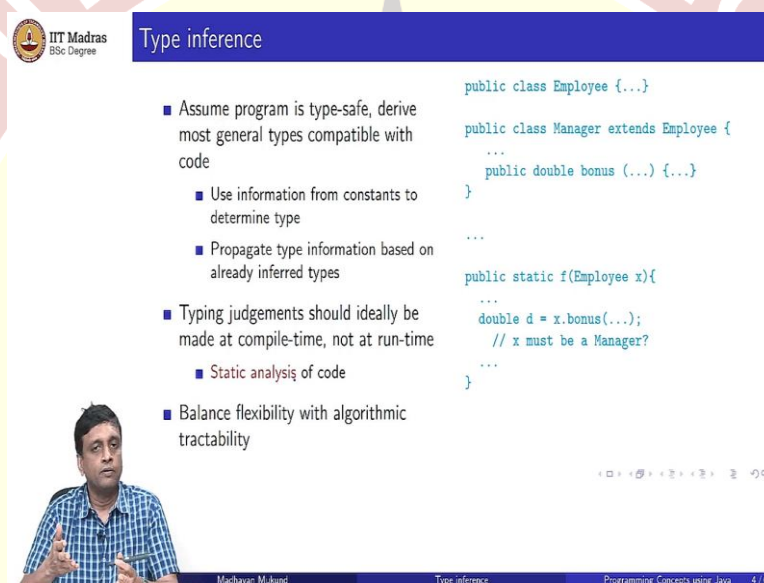
So, the real goal of this type inference is that each time I make a decision like this, I have created a kind of an assumption or an obligation saying that henceforth now, I have assumed that `x` is a Manager, so now, there is an obligation that every other use of `x` inside this thing must be compatible with Manager. So, I have created these type obligations because I have inferred something about the types from reading the code and now I must go through the rest of the code and keep this set of obligations.

So, it is like really solving a puzzle as I said. So, imagine you solve a Sudoku, you solve a Sudoku then you fill in some numbers. Now you go ahead assuming that the numbers you have filled in are correct. Now at some point you hit a wall. You say, I cannot put in the number I want, I want to put a 9 in this box; there is no place for a 9. Then you have to go back.

So, in type inference there is no going back. If you have to go back, then the program is wrong. You can only go forwards and if you finish the puzzle, if you are able to assign types to everything then the program is well typed. If at any point you got stuck because the information that you had used so far was inconsistent with what you need to do next, then you say, ok, there is some problem here.

So, type inference does not imply that the programmer does not annotate, it is only a saving thing. So, that if the programmer is allowed to be lazy and not write all the types then can the compiler figure out enough to make sure that all the types are statically assigned correctly.

(Refer Slide Time: 08:22)



IIT Madras
BSc Degree

Type inference

- Assume program is type-safe, derive most general types compatible with code
 - Use information from constants to determine type
 - Propagate type information based on already inferred types
- Typing judgements should ideally be made at compile-time, not at run-time
 - Static analysis of code
- Balance flexibility with algorithmic tractability

```
public class Employee {...}

public class Manager extends Employee {
    ...
    public double bonus (...) {...}
}

...

public static f(Employee x){
    ...
    double d = x.bonus(...);
    // x must be a Manager?
}

...
```

Madhavan Mukund

Type inference

Programming Concepts using Java 4/6

So, assuming that the program is typed safe, you want to derive the most general types. So, we use the constants and we use these inferred types to propagate. But, the crucial thing is that this must all be done statically. So, this is called static analysis in the sense that I read the code, so, this is the compiler doing it, it is not the Java runtime doing it.

So, when I read the code I must be able to make this judgment. And in order to make this judgment I need an algorithm which does this. So, I need to be able to read the code and keep track of all these dependencies and say, oh, this is a string and that is a string if that is a string and this is a Manager, if this is a Manager that is an Employee and so on.

And make sure at the end that everything is consistent with itself. So, the problem is to balance this flexibility. We are allowing the programmer to get away with less work, not having to declare everything. And on the other side we are putting an extra burden on the

compiler saying, now, the compiler has to fill in the gaps and make sure that there are no inconsistencies.

And we already discussed right at the beginning that in general this is a very hard problem. Program verification is actually unsolvable by a computer. We cannot write an algorithm which looks at a program and checks that it is doing what it is supposed to do. And this is towards that target.

So, we are trying to look at a problem and see whether the program and see whether it is well typed. It is a less difficult problem because it is more like consistency between types. And in some circumstances you can do it. So, depending on, how your types are organized, you can do it.

So, there are some languages, some functional languages in particular, where you do not have to write the types. But, there are then gray areas; there are some systems where you can write as a program which the compiler is not able to get the most general type. But, if you give annotations you can help it along. You can say I do not want the most general type.

In the previous case, it inferred something was an Employee or a Manager, you can say, no, no, I want to make sure it is a Manager, so, I changed that type so that you do not get confused. But in general, if you have a very flexible typing mechanism, like what we have with objects and this hierarchy and all that, this is not algorithmically possible. So, we have to really constrain this type inference to make it manageable.

(Refer Slide Time: 10:34)

IIT Madras
BSc Degree

Type inference in Java

- Java allows limited type inference
 - Only for local variables in functions
 - Not for instance variables of a class
- Use generic `var` to declare variables
 - Must be initialized when declared
 - Type is inferred from initial value
- Be careful about format for numeric constants
- For classes, infer most constrained type
 - `e` is inferred to be `Manager`
 - `Manager` extends `Employee`
 - If `e` should be `Employee`, declare explicitly

```
var b = false; // boolean
var s = "Hello, world"; // String
var d = 2.0; // double
var f = 3.141f; // float
var e = new Manager(...); // Manager
```

Employee
Manager

Madhavan Mukund

Type inference

Programming Concepts using Java 5/6

So, Java did not have any type inference when it was first created, but over time it has allowed a certain amount of type inference but in a very limited way. So, first of all Java can infer types in general only for local variables. So, remember that we have two types of variables that we use, we have these instance variables that are created a fresh with every class.

And then inside the functions that we write, we use these local variables which are used as temporary storage in order to compute whatever we need to compute. So, the variables that are defined inside the functions, for those you can use type inference. You cannot use it for the instance variables in a class. And how do you use this well? The way you use this is to not provide a type.

So, you have to still provide a declaration, like Java does not allow you to just like, it is not like Python, where if Python sees a new name x or y or name or string or anything, it will just say, oh, this is a new variable that the programmer has chosen to use, so let me allocate space for it based on the current value and proceed.

So, in Python, names are declared dynamically, in some sense, as they are encountered in the code and their value determines their type and the type can keep changing. So, Python has a very different approach towards both storage and typing, whereas in Java we need to first, know in advance, which values you are going to use and we need to know the type.

So, what we are now saying is that we need to know in advance which values are going to use but the type maybe you do not have to tell me. So, I just say that b is a variable. So, var b. Now, what is the type of b? Well, I initialize it to false. If I initialize it to false and this initialization expected to be correct, then the only type of variable that can be initialized to false is olean. So, therefore I can infer that b is a Boolean.

Similarly, if I take a string and I initialize it s and I initialize it to a string, then s must be a string because of constant initialization the string. So, now this is limited that question. So, earlier we said that you could take the type of some expression, used in assignment and then infer the type. Well, Java said says I do not want to become so complicated.

So, when you create a variable and initialize it at the time of declaration, you sometimes want to provide an initial value. Now, that initial value I can figure out the type. So, at that time if you do not want to tell me what a type is, then I will believe you and I will copy the type from the initialized value to the variable.

So, therefore this will become, s will become a string and b will become Boolean. So, it must be a local variable inside a function and it must be initialized at the time of declaration, then you can say var. Now, you now have to make sure that Java understands what you are writing. So, if I write 2.0 then for Java 2.0 on its own is always a double. I need an f at the end for a float. So, if I say d equal to 2.0, then d is actually a double.

You might think it is a float but it is not a float. Similarly this is clearly a float because I have written an f. So, if you want something to be of a particular type and you are relying on type inference, then make sure that you write the constant in such a way that there is no ambiguity about what type you are getting. For classes, the type is inferred in the most constrained way.

Remember that we had in our running example, we had Employee and we had Manager. So, in principle, e could be an Employee which is assigned to point to a Manager object because a super type can point to a subtype. But if I do not tell you what e is, if I say we, so this var extends to objects, it is not only for these simple types like numbers and Booleans and strings and so on. So, the var extends to objects, so, I can save var equal to new Manager.

So, now this is an, is effectively a declaration with an initialization. So, from this Java will allow me to then infer the type. So, it will say, ok, the type that I am going to infer is what you use on the right hand side. So, e is automatically a Manager, right. So, it is in some sense the most constrained type that Java can associate with that initialization.

Now if in this type hierarchy, there was a super type which you wanted e to be of, you wanted e to be actually an Employee and not a Manager, then it is your job to tell Java this. So, Java is not going to automatically assume that you wanted it to be something higher than Manager. If you wanted you to be an Employee, then by all means declare e to be an Employee, nobody is stopping you from doing this. But, if you wanted to infer the type it will only infer the most precise type that you can infer from the right hand side.

(Refer Slide Time: 15:07)



Summary

- Automatic type inference can avoid redundancy in declarations
Manager m = new Manager(...);
- Assuming the program is type-safe, derive most general types compatible with the code
 - Compiler can infer type from expressions used to assign values
 - Inferred type information can be propagated
- Challenge is to do this statically, at compile-time
- Java allows limited type inference
 - Only local variables that are initialized when they are declared



Madhavan Mukund

Type inference

Programming Concepts using Java 6 / 6

So, why would we do all this type inference? We will actually, it help us to avoid these kind of code which is sort of obviously redundant. If you say `Manager m = new Manager(...);` then it looks like you are providing tautological information or redundant information to the compiler and it is really well the compiler's job to figure out that these two things are not required.

One of them is enough. But on the other hand if you want to get around this then you have to make sure that the compiler has enough information to give you the right type. So, this is this type inference. So, type inference basically says assuming that the programmer is doing a good job.

So, assuming the program is type-safe then try to propagate the types. And this is through the expressions and this inferred values. But, the problem is that in general this is very algorithmically challenging, especially, if you have a very rich way of associating types, like in Java you can take a super type and assign it to be a subtype object and so on. This kind of thing makes it very difficult to do type inference in the full scale.

So, therefore Java has taken this attitude, the type inference is sometimes useful to limit the complexity of the syntax of the code, at the same time, it is too complicated to do in full generalities, like some functional programming languages. So, let us help the programmer by allowing local variables to be statically type inferred, provided the local variable is provided is given with the initialization which tells the Java compiler what time to choose, what type to choose. So, that is how type inference works in Java.